Coursework Submission Cover Sheet

Module	CC6004NM – Network and Cloud Security
Assessment No/Title	Assessment 2- Group Coursework
Assessment Weighting	50% of the module mark
Assessment Submission Deadline	9th October 2026
Module Leader	Mr. Akalanka Jayasinghe

Academic Integrity and Plagiarism

You are reminded that there exist regulations concerning plagiarism. Extracts from these regulations are printed below. Please sign below to say that you have read and understand these extracts

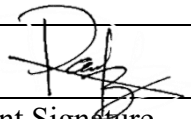
This header should be attached to the work you submit. No work will be accepted without it.
Extracts from University Regulations on
Cheating, Plagiarism, and Collusion
Section 2.3: "The following broad types of offense can be identified and are provided as indicative examples ….

I. Cheating: including taking unauthorized material into an examination; consulting unauthorized material outside the examination hall during the examination; obtaining an unseen examination in advance of the examination; copying from another examinee; using an unauthorized calculator during the examination or storing unauthorized material in the memory of a programmable calculator which is taken into the examination; copying coursework.
I. Falsifying data in experimental results.
II. Personation, where a substitute takes an examination or test on behalf of the candidate. Both the candidate and substitute may be guilty of an offense under these Regulations.
III. (Bribery or attempted bribery of a person thought to have some influence on the candidate's assessment.
IV. Collusion to present joint work as the work solely of one individual.
V. Plagiarism, where the work or ideas of another are presented as the candidate's own.
VI. Other conduct calculated to secure an advantage on assessment.
VII. Assisting in any of the above.

Some notes on what this means for students:

1. Copying another student's work is an offence, whether from a copy on paper or from a computer file, and in whatever form the intellectual property being copied takes, including text, mathematical notation and computer programs.
2. Text Box Taking extracts from published sources without attribution is an offence. To quote ideas, sometimes using extracts, is generally to be encouraged. Quoting ideas is achieved by stating an author's argument and attributing it, perhaps by quoting, immediately in the text, his or her name and year of publication, e.g. " e = mc2 (Einstein 1905)". A references section at the end of your work should then list all such references in alphabetical order of authors' surnames. (There are variations on this referencing system which your tutors may prefer you to use.) If you wish to quote a paragraph or so from published work then indent the quotation on both left and right margins, using an italic font where practicable, and introduce the quotation with an attribution.

Please sign below to say that you have read and understand these extracts:

	10/5/2026
Student Signature	Date



CC6004NM – Network and Cloud Security

Name: Group 2 – Based in Dubai / UAE

Date: Friday, 09 October 2026

First Supervisor: Mr. Akalanka Jayasinghe

Group A	
Name	LMU ID
Chathura Rankothge	**25040137**
Dalsha Kamith Balasooriya	**25040138**
Salah Uddin	**25040140**
Waleed Al Tulaihi	**25040142**
Bo Than Naing	**25040141**

1. Executive Summery

This project presents the design, implementation and evaluation of Mozard8, an educational symmetric cipher developed to demonstrate fundamental cryptographic concepts and their application to network communication. The cipher combines repeating-key byte shifts, position-dependent modular addition and XOR ciphertext chaining to transform plaintext into ciphertext. Unlike traditional alphabet-based ciphers, Mozard8 operates on UTF-8 bytes, allowing messages containing symbols and non-English characters to be processed.

The project implements Mozard8 in Python using separate encryption and decryption functions and a graphical user interface developed with Tkinter. TCP sockets facilitate message transmission between a Windows computer and a Kali Linux virtual machine, while Wireshark enables inspection of network traffic. Testing covers encryption and decryption correctness, character compatibility, input validation, network transmission and ciphertext observation.

The evaluation identifies important limitations, including deterministic encryption, repeating-key weaknesses and the absence of integrity and authentication mechanisms. Although the application demonstrates successful encrypted communication when the sender and receiver use the same key and compatible implementations, unreadable network payloads alone do not establish cryptographic security. Mozard8 is therefore intended for educational demonstration rather than protecting sensitive information. The project provides practical insight into cipher design, Python implementation, network communication and the distinction between data obfuscation and security provided by established cryptographic standards.

Table of Contents

1. Introduction - Target words 250 7
1.1. Context 7
1.2. Project Problem 7
1.3. Project aim 7
1.4. Objectives 7
2. Background – 180 Words 8
3. Existing Ciphers 9
3.1. Ceaser Cipher 9
3.2. Affine Cipher 9
4. Comparative Analysis 11
5. Design Requirements – 180 words 12
5.1. Functional requirements 12
5.2. Non-functional requirements 12
6. Mozard8 Cipher Design – 350 words 13
7. Encryption Algorithm – 220 words 14
8. Decryption Algorithm – 220 wors 15
9. System Design – 130 words 16
10. Implementation – 150 words 17
11. User Interface – 100 words 18
12. Testing – 220 words 19
13. Security Evaluation – 220 words ..... **Error! Bookmark not defined.**
14. Strengths & Weaknesses – 120 words 20
15. Applications – 100 words 22
16. Teamwork – 100 words 23
17. Limitations & Future work - 100 24
18. Conclusion – 100 26
19. References 27
20. Appendix A — Full Python Source Code 28
21. Appendix B — Detailed Encryption Output 32
22. Appendix C — Detailed Decryption Output 33
23. Appendix D — Test Results 34
24. Wireshark Network Capture 35

Figure 1Full pthon code - Ciper.py 28
Figure 2 Full python code - gui.py 30
Figure 3 Detailed Encryption Output 32
Figure 4 Detailed Decryption Output 33
Figure 5 Wireshark capture showing the encrypted ciphertext transmitted from Windows to Kali over TCP port 5000 35

2. Introduction

2.1. Context

Cryptography safeguards information using mathematical methods. Encryption transforms readable plaintext into ciphertext, and decryption recovers the original message with the correct key (Stinson & Paterson, 2018, p. 1).. These processes protect stored data and communications from unauthorized disclosure.

The National Institute of Standards and Technology (NIST) identifies confidentiality as a primary service provided by cryptographic algorithms (Evans, et al., 2001, p. 1) Cryptography also facilitates digital investigations, where cryptographic hashes preserve evidence authenticity (Oard & Webber, 2013, p. 114). Understanding of encryption and decryption is essential for examining information protection and digital evidence analysis.

2.2. Project Problem

Classical ciphers illustrate foundational cryptographic concepts but exhibit well-documented vulnerabilities. The Caesar cipher, offers only 26 alphabetic shifts, enabling exhaustive key searches. Affine substitution ciphers are susceptible to frequency analysis, and the Vigenère cipher's repeating key enables statistical cryptanalysis (Stinson & Paterson, 2018, pp. 17-19 ,40-48). Transposition ciphers alter characters rather than substituting values (Stinson & Paterson, 2018, pp. 32-33).

These limitations JUSTIFY exploring combined transformations, while recognizing that combining basic operations does not provide modern cryptographic security.

2.3. Project aim

This project design, implement, and evaluate Mozard8, an educational symmetric cipher that integrates repeating-key byte shifts, position-dependent modular addition, and XOR ciphertext chaining. A Python application demonstrates encrypted TCP communication between a local computer and a Kali Linux virtual machine, with network traffic analyzed using Wireshark.

2.4. Objectives

I. To analyse selected classical ciphers and their limitations.
II. To identify design principles informing Mozard8.
III. To specify reversible encryption and decryption algorithms.
IV. To implement Python application supporting UTF-8 messages and network transmission.
V. To test correctness, compare captured traffic, and evaluate security limitations and educational suitability.

3. Background

Cryptography establishes the basis for safeguarding digital information through key-controlled transformations. Contemporary encryption standards exemplify how these principles ensure confidentiality. For example, the Advanced Encryption Standard (AES) processes 128-bit blocks using keys of 128, 192, or 256 bits (Evans, et al., 2001, p. 2) In contrast to educational prototypes, AES includes a published specification defines its transformations and implementation requirements.

Digital investigations must address both evidence authenticity and confidentiality. In e-discovery, preservation entails maintaining the original bit stream, retaining interpretive information, and ensuring authenticity. Cryptographic hashes facilitate this by providing reference for detecting alterations (Oard & Webber, 2013, p. 114).Encryption and hashing fulfil distinct investigative functions and should not be considered interchangeable.

Within this context, Mozard8 serves as a practical platform for exploring reversible encryption. Its architecture integrates repeating-key byte shifts, position-dependent modular addition, and XOR ciphertext chaining. Implementation in Python and analysis using Wireshark link the algorithm network communication. The project assesses successful message recovery and limitations, such as deterministic output and lack of integrity verification, emphasizing that unreadable traffic does not equate to demonstrated security.

4. Existing Ciphers

4.1. Ceaser Cipher

The Caesar cipher is a substitution cipher in which each letter is shifted by a fixed number of positions. When letters are represented as integers from zero to twenty-five, the encryption is:

$$C = (P + k) \, mod \, 26$$

Decryption subtracting the same key. Its advantages are simplicity, low computational cost, clearly defined inverse. However, it has only twenty-six possible keys, making exhaustive search trivial (Stinson & Paterson, 2018, pp. 17-19). The principle of modular addition in the shift cipher directly informs the design of Mozard8, which operates on byte values modulo 256.

3.3 Vigenère Cipher

The Vigenère cipher uses a repeating keyword to apply different alphabetic shifts:

$$Ci = (Pi + ki \, mod \, l) \, mod \, 26$$

It applies different shifts successive positions, allowing identical plaintext letter to produce different ciphertext (Li, et al., 2015, p. 129) explain this repeating-key process using VIGvisual. Its strength transparent polyalphabetic structure, but repeating keys introduces patterns exploitable through Kasiski examination and index of coincidence (Li, et al., 2015, pp. 131-132) .

Vigenère directly inspires Mozard8's repeating key-byte shifts, extended to bytes modulo 256 and incorporates position. These modifications do not remove repeating key vulnerabilities.

4.2. Affine Cipher

Affine encryption combines multiplication and addition:

$$C = (aP + b) \, mod \, 26$$

The multiplier must be relatively prime to twenty-six to ensure a modular inverse. It offers mathematical clarity and straightforward implementation. (Kasianchuk, et al., 2025, p. 4) explain the reversible transformation, and highlights its vulnerability (Kasianchuk, et al., 2025, pp. 1-2).

Fixed substitution preserves frequency patterns, and the twelve valid multipliers with twenty-six offsets yield only 312 keys (Stinson & Paterson, 2018, pp. 22-25) .Mozard8 employs modular arithmetic but not affine multiplication (Kasianchuk, et al., 2025)

3.3 Transposition Cipher

Transposition alters characters positions without modifying values. The key determines rearrangement within each block, and decryption uses the inverse permutation (Stinson & Paterson, 2018, p. 32).

It conceals messages by altering order but preserves character frequencies, leaving patterns detectable. Mozard8 does not employ transposition, XOR chaining preserves positions while combining bytes values. Transposition serves as contrasting method rather than a design element.

5. Comparative Analysis

The four ciphers demonstrate distinct reversible transformations. Comparative analysis reveals educational features and clarifies which techniques contribute to Mozard8.

Cipher	Principal operation	Strength	Limitation	Influence on Mozard8
Caesar	Fixed modular shift	Simple implementation	Small key space.	Foundation for additive byte shifts (Stinson & Paterson, 2018, pp. 17-19)
Vigenère	Repeating keyword shifts	Varying substitutions	Repetition enables cryptanalysis	Inspiration for repeating key bytes (Li, et al., 2015, pp. 129-132)
Affine	Modular multiplication and addition	Mathematical reversibility	Preserve frequency patterns	Supports modular-arithmetic concepts, (Kasianchuk, et al., 2025, pp. 1-2,4).
Transposition	Rearrangement of positions	Demonstrates permutation	Preserves character counts	Contrasting technique (Stinson & Paterson, 2018, pp. 32-33).

Table 1 Comparative Analysis table

Mozard8 extends repeating-key addition to UTF-8 bytes, incorporates byte's position, and applies XOR feedback. These operations enable distinct processing stages and straightforward decryption. However, can be eliminate by XORing adjacent ciphertext bytes, and subtracting position reveals a repeating-key additive relationship. These additions do not provide strong protection against cryptanalysis.

Mozard8 is justified by educational transparency rather than production security. Testing should confirm accurate recovery, Unicode compatibility, and predictable error handling, while evaluation should acknowledge deterministic output and the lack of authentication. Unreadable Wireshark payloads indicate data transformation rather than resistance to attack.

6. Design Requirements

The aim of this project was to design and implement an original symmetric cipher, ***Mozard8***, and to demonstrate it protecting a message sent between two machines on a network. The following requirements guided the design.

6.1. Functional requirements

- Encrypt any text message into ciphertext using a secret key and decrypt it back to the exact original text with the same key.
- Accept a user-chosen key of any length greater than zero.
- Support all characters, including symbols and non-English text, by working on bytes instead of only A to Z.
- Send the message from a local PC to a Kali Linux virtual machine over TCP so the traffic can be captured in Wireshark.
- Provide a graphical interface so the cipher can be used without the command line.

6.2. Non-functional requirements

- The same letter must not always produce the same ciphertext byte, so simple patterns are hidden.
- Ciphertext must be the same length as the plaintext, keeping the network payload easy to inspect.
- The program must run on Windows and Linux using only the Python standard library.
- Invalid input must produce a clear error message instead of a crash.

7. Mozard8 Cipher Design

Mozard8 is a symmetric cipher: the sender and receiver share one secret key, and the same key is used to encrypt and decrypt. It processes the message one 8-bit byte at a time, so every value it between 0 and 255 and all arithmetic is done modulo 256. Working on bytes means any character can be supported, because the text is first converted to UTF-8.

The design combines two classical ideas.

7.1. Position-dependent shift

Each plaintext byte is shifted by adding a key byte and the byte's position in the message. The key repeats along the message, as in the Vigenère cipher, but adding the position means the shift keeps changing even when the key repeats. A key of length 11 would normally reuse the same shift every 11 characters; in Mozard8 it does not.

7.2. Ciphertext chaining

The shifted byte is combined, using XOR, with the previous ciphertext byte. Each output depends on everything encrypted before it, and repeated letters in the plaintext produce different ciphertext bytes. The first byte has no predecessor, so a starting value is derived from the key by adding all its bytes together modulo 256.

The cipher runs as six processing stages Mozard8

1. **Encode**: convert the message text to UTF-8 bytes.
2. **Key preparation**: convert the key to bytes and reject an empty key.
3. **Starting value**: sum the key bytes modulo 256.
4. **Shift**: add the key byte and position to the plaintext byte.
5. **Chain**: XOR the result with the previous ciphertext byte.
6. **Feedback**: store the new ciphertext byte as the "previous" value for the next byte and append it to the output.

Stages 4 to 6 repeat for every byte.

Limitations. Mozard8 was designed for learning and demonstration and is not suitable for protecting real data. An eavesdropper can see every previous ciphertext byte, so the chaining step can be undone without the key, leaving a Vigenère-style cipher that can be attacked with frequency analysis. The cipher uses no random value, so the same message and key always give the same ciphertext, and it has no integrity check. Production systems should use a vetted algorithm such as AES.

8. Encryption Algorithm

Encryption takes a plaintext string and key and returns ciphertext bytes of the same length.

Notation: P[i] is the plaintext byte at position i (starting from 0), K is the key as bytes with length L, and C[i] is the ciphertext byte.

Steps

Convert the plaintext and the key to UTF-8 bytes. If the key is empty, stop with an error.

Calculate the starting value: prev = (sum of all key bytes) mod 256.

For each plaintext byte P[i]:

- shifted = (P[i]+κ [mod L] + i) mod 256
- C[i] = shifted XOR prev
- prev =C[i]

Return all C[i] bytes as the ciphertext.

Worked example with the key "Mozard8" and the message "Hello". The key bytes add up to 1109, and 1109 mod 256 = 85, so the starting value is 85 (hex 55).

i	Letter	P[i]	Key byte	Shifted	prev	C[i]
0	H	72	77	149	85	30
1	e	101	121	223	30	e5
2	l	108	83	193	e5	0d
3	l	108	97	208	0d	dd
4	o	111	114	22*	dd	38

Table 2 Encryption Algorithm table

The full ciphertext for "Hello" is ***30 e5 0D dd 38***, matching the program output and confirming that repeated letters encrypted differently.

9. Decryption Algorithm

Decryption reverses the two steps applied during encryption. It takes ciphertext bytes and the same key and returns the original plaintext.

Notations: C[i] is the ciphertext byte at position **I, K** is the key as bytes with Length **L**, and **P[i]** is the recovered plaintext byte.

Steps

Convert the key to UTF-8 bytes. If the key is empty, stop with an error.

Calculate the same starting value $\text{prev} = (\text{sum of all key bytes}) \bmod 256$

For each ciphertext byte C[i]:

$$\text{shifted} = \text{C[i] XOR prev}$$

$$\text{P[i]} = (\text{shifted} - \text{K [i mod L]} - \text{i}) \bmod 256$$

$$\text{prev} = \text{C[i]}$$

store each recovered byte and decode the final array as UTF-8

Worked example with the key "Mozard8" and the ciphertext **30 e5 0d dd 38**.

i	C[i]	prev	shifted	Key byte	P[i]	Letter
0	30	85	0xB5	77	72	H
1	e5	30	0xD5	121	101	e
2	0d	1f	0xE8	83	108	l
3	dd	0d	0xD0	97	108	l
4	38	dd	0xE5	114	111	o

Table 3 Decryption Algorithm

The recovered plaintext is **"Hello"**, confirming that XOR chaining and position-dependent shifts are fully reversible when the correct key is used

10. System Design

The system follows a linear pipeline from the user's message to the ciphertext placed on the network.

10.1. User Input

The user ENTERS a message, a secret key and the receiver's IP address and port number in the graphical interface.

10.2. Input Validation

The program checks that the message and IP address are not empty and verifies that the port is a number from 1 to 65535, The key must contain at least one character.

10.3. Key Preparation

The key text is converted to UTF-8 bytes. The starting value for chaining is computed by summing all key bytes modulo 256. No random values are used; the user supplies the entire secret.

10.4. Encryption Engine:

The encrypt () function receives the plaintext and key and applies Mozard8's reversible byte-based transformations.

10.5. Mozard8 Processing Stages

The six stages described earlier -encoding, key preparation, stating value, position -dependent shift, XOR chaining, and feedback- are applied to every bite.

10.6. Cyphertext Transmission

The resulting ciphertext bytes are sent over a TCP connection. The receiver, which reverses applies decrypt () to reverse the process and displays the recovered plaintext.

11. Implementation

Mozard8 is implemented in Python 3 (version 3.8 or later) and uses only the standard library, so no external packages are required. The program is split into modules. `cipher.py and gui.py.`

The “cipher.py” module contains the encryption and decryption the algorithms. The `_key_bytes ()` helper converts the user-provided key into UTF-8 bytes and raises ‘ValueError” if the key is empty. The encrypt () function applies position-dependent modular addition and XOR chaining to the plaintext byte, producing ciphertext of equal length. The ‘decrypt()’ function reverses these operation by undoing the XOR step and subtracting the key byte and position, restoring the original message.

The gui.py module implements a graphical interface with Tkinter and manages network communication through TCP sockets. Sending and receiving occur background threads, while queue transfers network events to interface safely. The application validates port numbers, blocks empty messages from being sent, and reports network errors through the activity log. This GUI operates on both Windows and Kali Linux, allowing encrypted messages to be exchanged and subsequently inspected using Wireshark.

12. User Interface

The program provides a single desktop window built with Tkinter, and the same interface runs on both machines. It is divided into four functional areas.

12.1. Settings

A port number is provided along with button to start or stop listening. Incoming data is process processes in a background thread and delivered to the interface through a queue.

12.2. Receive

A port number is provided along with a button to start or stop listening. Incoming data is processed in a background thread and delivered to the interface through a queue.

12.3. Send

The user specifies the destination IP address, port, and message. Pressing Send transmits either plaintext or ciphertext depending on the encryption setting

12.4. Activity Log

A color-coded log displays the plaintext, the hexadecimal bytes sent on the wire, and the decrypted message on the receiving side. Network errors and invalid input are shown either in a message box or in red within the log.

13. Testing

Testing was carried out at three levels.

13.1. Unit testing

Nine automated tests in `test_cipher.py` were run with `python -m unittest`, and all nine passed. They check that:

- Decrypting an encrypted message returns the original text;
- ***"Hello" with the key "Mozard8" produces the expected ciphertext 30e50ddd38;***
- The ciphertext differs from the plaintext and the same length;
- Repeated letters produce different ciphertext bytes;
- The wrong key does not recover the message;
- Symbols and non-English characters survive a round trip;
- An empty message is handled, and an empty key is rejected with an error.

Functional testing. The interface was tested manually:

Test	Expected result	Result
Send with encryption on	Receiver shows the original message	Pass
Send with encryption off	Receiver shows plaintext	Pass
Receivers use a different key	Unreadable output	Pass
Send when no receiver is listening	"Connection refused" error shown, no crash	Pass
Port set to letters	Error message shown	Pass

Table 4 Functional testing table

13.2. Network testing

Messages were sent from the local PC to the Kali VM while Wireshark captured traffic with the filter `tcp.port == 5000`. In the plaintext, run the message was readable in the TCP stream. In the encrypted run, the payload was unreadable, and its hexadecimal bytes matched the ciphertext shown in the sender's log (Appendix E).

14. Security Evaluation

Mozard8 was evaluated using functional testing, brute-force experiments, known-plaintext analysis and inspection of captured network traffic. In one test, the key *Mozard8* encrypted the message ***Hello as 30e50ddd38.*** The key bytes sum to 1109, giving an initial chaining value of 85 (0*55). This test is separate from the earlier functional-testing example that used a different key.

The main weakness of Mozard8 comes from its repeating-key design. For every byte after the first, an observer can reverse the chaining step by XORing each ciphertext byte with the one before it. Subtracting the byte position then reveals a byte-level Vigenère-style transformation.

If an attacker knows a plaintext byte P_i, they can calculate the corresponding key byte:

$$K_i = (V_i - P_i)\,256$$

where:

$$V_i = \Big(\big((C_i \oplus C_{i-1}) - i\big)\,256\Big)$$

Because the key repeats, enough known plaintext can reveal the entire key. This makes frequency analysis and Kasiski-style attacks applicable.

During the brute-force experiment, the lowercase three-character key cat was found at candidate 1,372 out of 17,576 possibilities. A separate 16-byte known-plaintext test recovered all 16 key positions. These positions matched the repeating seven-byte key and exposed the complete key. Scripts and execution evidence are included in Appendix E.

A longer key increases the brute-force search space: each additional lowercase character multiplies the possibilities by 26, while each additional arbitrary byte multiplies them by 256. However, this does not prevent the demonstrated known-plaintext attack. Wireshark inspection should be supported by a specific observation, such as hexadecimal TCP payload values matching the ciphertext shown in the sender's log.

Overall, the results show that Mozard8 is useful for demonstrating encryption, decryption and unreadable network traffic. But not show that it is suitable for protecting sensitive communications

15. Strengths & Weaknesses

Mozard8 has several educational strengths. Its modular arithmetic and XOR operations are simple to implement and clearly demonstrate reversible byte-level transformations. Processing UTF-8 supports a wider range characters than alphabet-only ciphers. The Tkinter interface, TCP communication and Wireshark inspection link cryptographic concepts with practical network analysis showing how ciphertext appears on the wire. The modular structure also supports testing and incremental developments

However, Mozard8 has security weaknesses. Its repeating-key creates exploitable relationships, and deterministic encryption produces identical ciphertext for identical inputs The cipher provides authentication and cannot detect ciphertext manipulation. Known-plaintext analysis can recover key bytes, and short keys are vulnerable to brute-force attacks. Mozard8 is suitable for teaching, not protecting sensitive data

16. Applications

Mozard8 provides educational demonstrations in network security, introductory cryptography and secure software development. Students can examine modular arithmetic, repeating-key behaviors, XOR chaining and the relationship between encryption and decryption. The Python application also enables practical exercises involving graphical interfaces, TCP sockets, error handling and cross-platform communication between Windows and Kali Linux. Wireshark captures allow learners to compare plaintext with transformed payloads and observe how application data appears during network transmission. The project may also serve as a starting point for experiments involving cryptanalysis and authenticated encryption. However, Mozard8 is unsuitable for protecting confidential information in real-world applications.

17. Teamwork

Session	Team Member Name	Main Responsibility	Report Sections	Practical Work
Session 1	Bo Than Naing	Research Existing Cipher	1. Introduction 2. Background 3.Existing Ciphers	Study Ceaser Cipher, Vigenère, Affine and Transposition ciphers.
Session 2	Kamith Balasooriya	Design the New Cipher	4.Comparative Analysis 5.Design Requirements 6.Cipher Design	Finalize Mozard8 structure, key, alphabet, formulas and stages.
Session 3	Waleed Al Tuliaihi	Encryption Algorithm	7. Encryption Algorithm part of 9. Encryption Walkthrough	Develop Python encryption code and produce step-by-step encryption output.
Session 4	Salah Uddin	Decryption Algorithm	8.Decryption Algorithm part of 10.Decryption Walkthrough	Develop Python decryption code and demonstrate recovery of plaintext.
Session 5	Chathura / Kamith	Testing Security & Evaluation	11. System Design 14. Testing 15.Security Evaluation 16.Strengths & Weaknesses	Run test cases, key tests, edge cases and collect screenshots/results
Session 6	Chathura	Application and Team Work Final Report	13. User Interface 17. Applications 18.Teamwork 19.Limitations/Future Work 20. Conclusion	Final integration, report formatting, presentation preparation

Table 5 Teamwork

The project was structured into six sessions, with responsibilities assigned according to primary development tasks. Bo Than Naing conducted research on existing ciphers. Kamith Balasooriya contributed to the comparative analysis, design requirements, and the Mozard8 design. Waleed Al Tuliaihi developed the encryption algorithm, while Salah Uddin managed decryption. In Session 5, Chathura and Kamith focused on system design, testing, and security evaluation. Session 6 involved Chathura coordinating application documentation, report integration, formatting, and preparation of the presentation. The session tracker documents both planned responsibilities and actual activities. Verification of task completion and individual contributions should be based on the group's documented work and supporting evidence provided in Appendix F.

18. Limitations & Future work

Mozard8 functions as an educational cipher rather than a cryptographically validated algorithm. Its repeating-key structure and deterministic encryption reveal ciphertext patterns, and XOR chaining relies on previously transmitted ciphertext bytes that are observable by an eavesdropper. Future development should replace the repeating-key method with a secure key-derivation mechanism and incorporate a random nonce or initialization vector to enable non-deterministic encryption. The application also lacks integrity and authentication; implementing a message authentication code or authenticated encryption would address this deficiency. Additional testing is recommended to assess these enhancements. For production environments, substituting Mozard8 with AES-GCM would offer a more robust foundation while preserving the current graphical user interface and Wireshark demonstration framework.

19. Conclusion

Mozard8, an original educational symmetric cipher, was designed, implemented, and evaluated in this project. The cipher integrates repeating-key byte shifts, position-dependent modular addition, and XOR ciphertext chaining to convert plaintext into ciphertext. A Python-based graphical application was developed with Tkinter and TCP sockets to facilitate encrypted communication between Windows and Kali Linux systems. Functional testing and Wireshark analysis demonstrated that messages could be encrypted, transmitted, and accurately decrypted at the receiving endpoint. Despite these capabilities, Mozard8 exhibits significant security limitations, such as the use of a repeating key, deterministic encryption, and the lack of authentication and integrity protection. Consequently, Mozard8 is appropriate for educational demonstration purposes rather than real-world secure communication.

20. References

Evans, D., Brown, K. & Bond, P., 2001. FIPS 197 Federal Information Processing Standards Publication Advanced encryption Standard (AES). *Advanced Encryption Standard (AES),* 26 11.pp. 1-46.

Kasianchuk, M. et al., 2025. Affine Cipher Encryption Technique Using Residue Number System. *Cryptography,* 24 04, 9(2), p. 26.

Li, C. et al., 2015. *A Visualization Tool for the Vigenère Cipher.* Vilnius, s.n., pp. 129-134.

Oard, D. W. & Webber, W., 2013. Information retrieval for e-discovery. *Foundations and Trends in Information Retrieval,* 26 06, 7(2-3), pp. 99-237.

Stinson, D. & Paterson, M. . B., 2018. *Introduction to Cryptography: Theory and Practice.* 4th ed. New York: Informa.

21. Appendix A — Full Python Source Code

Ciper.py

gui.py cipher.py

```python
"""
cipher.py - a simple custom symmetric cipher for a classroom demo.

NOT secure for real use. It is a homemade design meant to show how
encryption and decryption work.

How it works (per byte, position i):
  1. SHIFT:  add the key byte and the position to the plaintext byte (mod 256)
             -> like Vigenere, but on bytes, and the shift changes with position
  2. CHAIN:  XOR the result with the previous ciphertext byte
             -> the same letter encrypts differently each time it appears

Decryption does the same two steps in reverse order.
"""


def _key_bytes(key: str) -> bytes:
    kb = key.encode("utf-8")
    if not kb:
        raise ValueError("Key must not be empty")
    return kb


def encrypt(plaintext: str, key: str) -> bytes:
    kb = _key_bytes(key)
    prev = sum(kb) % 256          # starting value for the chain, derived from the key
    out = bytearray()
    for i, p in enumerate(plaintext.encode("utf-8")):
        shifted = (p + kb[i % len(kb)] + i) % 256   # step 1: shift
        c = shifted ^ prev                          # step 2: chain
        out.append(c)
        prev = c
    return bytes(out)


def decrypt(ciphertext: bytes, key: str) -> str:
    kb = _key_bytes(key)
    prev = sum(kb) % 256
    out = bytearray()
    for i, c in enumerate(ciphertext):
        shifted = c ^ prev                          # undo step 2
        p = (shifted - kb[i % len(kb)] - i) % 256    # undo step 1
```

Figure 1Full pthon code - Ciper.py

```
40      for i, c in enumerate(ciphertext):
41          shifted = c ^ prev                          # undo step 2
42          p = (shifted - kb[i % len(kb)] - i) % 256    # undo step 1
43          out.append(p)
44          prev = c
45      return out.decode("utf-8", errors="replace")
46
47
48  if __name__ == "__main__":
49      # Standalone demo: python3 cipher.py
50      key = "Mozard8"
51      message = "Hello, this is a secret message!"
52      ct = encrypt(message, key)
53      print("Key        :", key)
54      print("Plaintext  :", message)
55      print("Ciphertext :", ct.hex())
56      print("Decrypted  :", decrypt(ct, key))
57
```

gui.py cipher.py

```python
"""
gui.py - desktop UI for the cipher demo. Run the same file on both machines.

  python gui.py        (Windows / macOS)
  python3 gui.py       (Kali; if tkinter is missing: sudo apt install python3-tk)

Needs cipher.py in the same folder.
"""
import queue
import socket
import threading
import tkinter as tk
from tkinter import messagebox, scrolledtext, ttk

from cipher import decrypt, encrypt


class App:
    def __init__(self, root):
        self.root = root
        self.events = queue.Queue()      # network threads -> UI thread
        self.server = None
        root.title("Cipher Demo")
        root.minsize(640, 520)

        pad = {"padx": 6, "pady": 4}

        # --- settings ---
        top = ttk.LabelFrame(root, text="Settings")
        top.pack(fill="x", **pad)
        ttk.Label(top, text="Secret key:").grid(row=0, column=0, sticky="w", **pad)
        self.key = tk.StringVar(value="Mozard8")
        ttk.Entry(top, textvariable=self.key, width=28).grid(row=0, column=1, sticky="w", **pad)
        self.use_cipher = tk.BooleanVar(value=True)
        ttk.Checkbutton(top, text="Encrypt / decrypt messages",
                        variable=self.use_cipher).grid(row=0, column=2, columnspan=2, sticky="w", **pad)

        # --- receive ---
        rx = ttk.LabelFrame(root, text="Receive")
        rx.pack(fill="x", **pad)
        ttk.Label(rx, text="Listen on port:").grid(row=0, column=0, sticky="w", **pad)
        self.listen_port = tk.StringVar(value="5000")
```

Figure 2 Full python code - gui.py

gui.py cipher.py

```
        ttk.Entry(rx, textvariable=self.listen_port, width=8).grid(row=0, column=1, sticky="w", **pad)
        self.listen_btn = ttk.Button(rx, text="Start listening", command=self.toggle_listen)
        self.listen_btn.grid(row=0, column=2, **pad)
        self.status = ttk.Label(rx, text="Not listening")
        self.status.grid(row=0, column=3, sticky="w", **pad)

        # --- send ---
        tx = ttk.LabelFrame(root, text="Send")
        tx.pack(fill="x", **pad)
        tx.columnconfigure(1, weight=1)
        ttk.Label(tx, text="Send to IP:").grid(row=0, column=0, sticky="w", **pad)
        self.host = tk.StringVar(value="127.0.0.1")
        ttk.Entry(tx, textvariable=self.host, width=18).grid(row=0, column=1, sticky="w", **pad)
        ttk.Label(tx, text="Port:").grid(row=0, column=2, sticky="e", **pad)
        self.send_port = tk.StringVar(value="5000")
        ttk.Entry(tx, textvariable=self.send_port, width=8).grid(row=0, column=3, sticky="w", **pad)
        ttk.Label(tx, text="Message:").grid(row=1, column=0, sticky="w", **pad)
        self.message = tk.StringVar()
        entry = ttk.Entry(tx, textvariable=self.message)
        entry.grid(row=1, column=1, columnspan=2, sticky="ew", **pad)
        entry.bind("<Return>", lambda e: self.send())
        ttk.Button(tx, text="Send", command=self.send).grid(row=1, column=3, **pad)

        # --- log ---
        logf = ttk.LabelFrame(root, text="Activity")
        logf.pack(fill="both", expand=True, **pad)
        self.log = scrolledtext.ScrolledText(logf, height=14, state="disabled",
                                             font=("Courier", 10), wrap="word")
        self.log.pack(fill="both", expand=True, padx=4, pady=4)
        self.log.tag_config("sent", foreground="#1a5fb4")
        self.log.tag_config("recv", foreground="#26a269")
        self.log.tag_config("err", foreground="#c01c28")
        ttk.Button(logf, text="Clear", command=self.clear_log).pack(anchor="e", padx=4, pady=(0, 4))

        root.protocol("WM_DELETE_WINDOW", self.close)
        root.after(100, self.poll)

    # ---------- helpers ----------
    def write(self, text, tag=None):
        self.log.config(state="normal")
        self.log.insert("end", text + "\n", tag)
        self.log.see("end")
```

22. Appendix B — Detailed Encryption Output

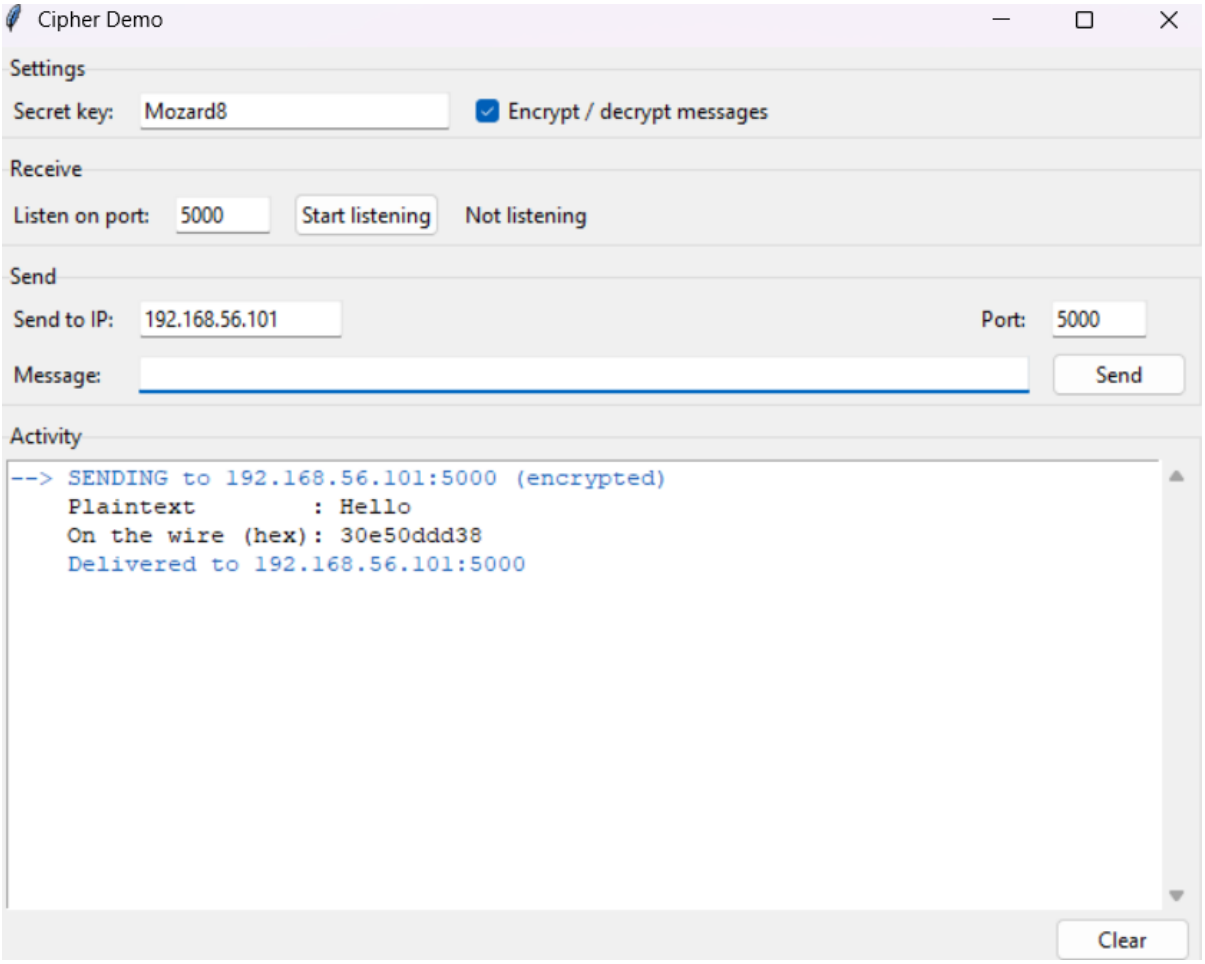


Figure 3 Detailed Encryption Output

23. Appendix C — Detailed Decryption Output

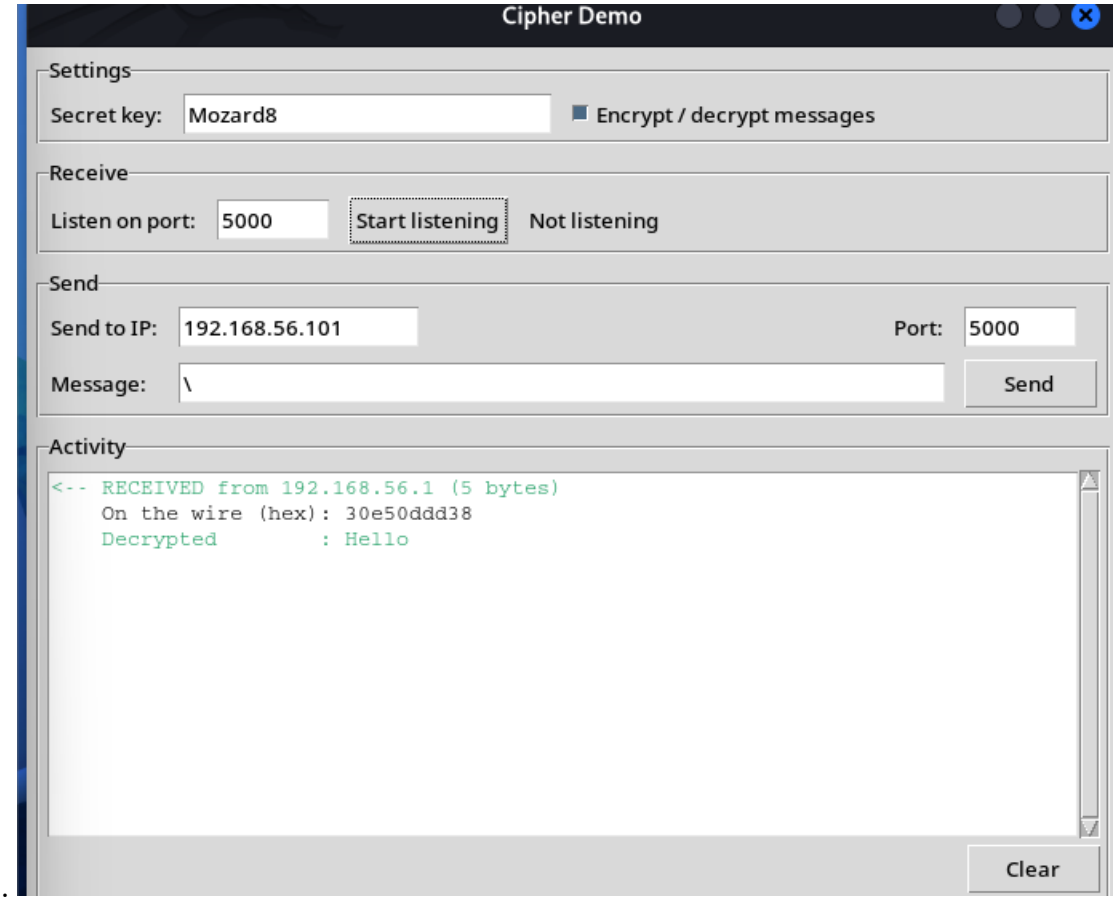


Figure 4 Detailed Decryption Output

24. Appendix D — Test Results

Table 6 Test Results

Test	Test Description	Input	Key	Expected Results	Actual Results	Status
TO1	Basic encryption	Hello	Mozard8	Cipher text Generated	30e50ddd38 generated	Pass
TO2	Network Transmission	Hello Windows → Kali	Mozard8	Encrypted data transmitted over TCP 5000	30e50ddd38 transmitted successfully	Pass
TO3	Decryption	30e50ddd38	Mozard8	Hello recovered	Hello recovered	Pass
TO4	End-to-end communication	Windows → Kali	Mozard8	Message received and decrypted	Hello received and decrypted	Pass
TO5	Empty input	Empty message	Mozard8	Message should not transmit	Transmission prevented	Pass
TO6	Special characters	Hello@123!	Mozard8	Message encrypted and correctly recovered	Original message recovered	Pass
TO7	Incorrect Key	Valid ciphertext + Incorrect key	Incorrect Key	Original plain text not recovered	Original plaintext not recovered	Pass
TO8	Reverse Communication	Kali → Windows	Mozard8	Message received and decrypted	Successfully received and decrypted	Pass
TO9	Wireshark verification	TCP port 5000	Mozard8	Ciphertext visible in network payload	Ciphertext bytes observed	Pass

25. Appendix E - Wireshark Network Capture

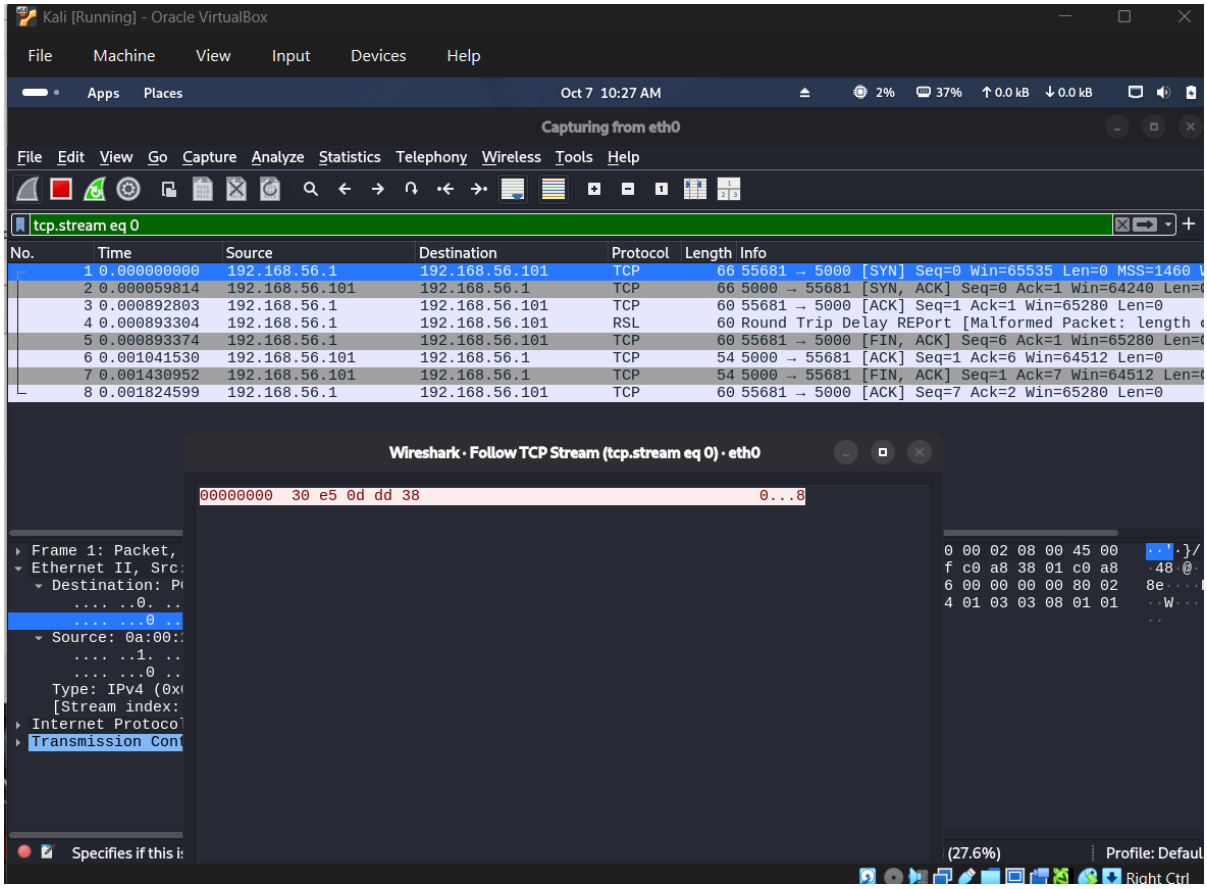


Figure 5 Wireshark capture showing the encrypted ciphertext transmitted from Windows to Kali over TCP port 5000

26. Appendix - F

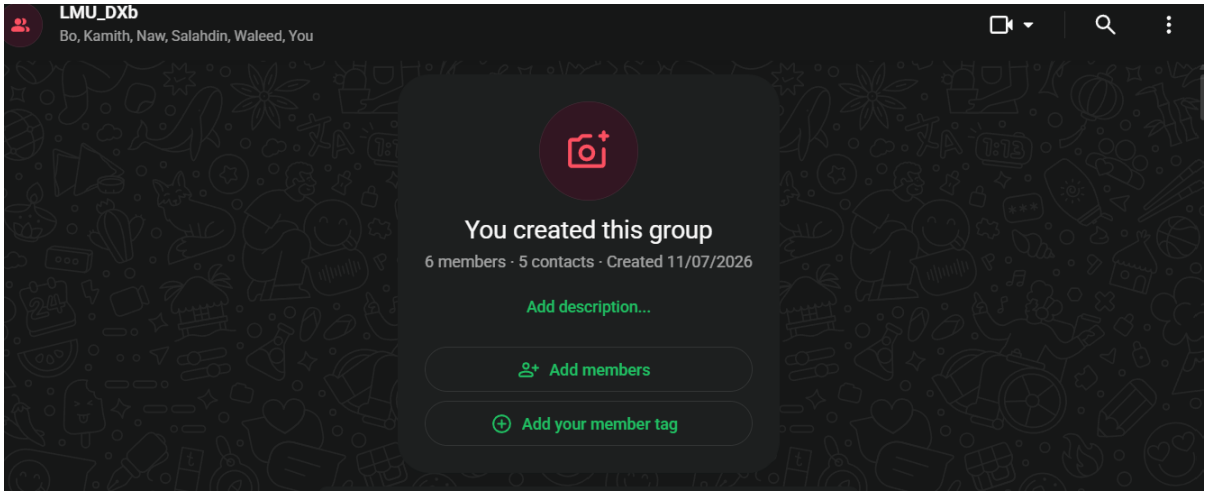
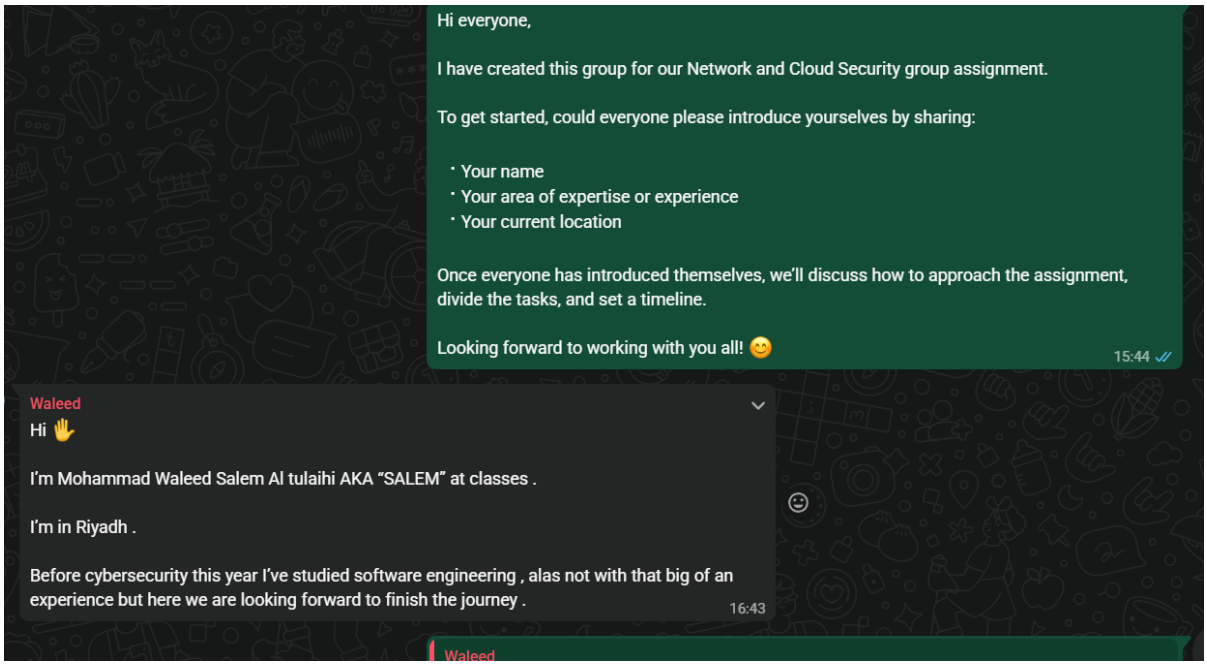
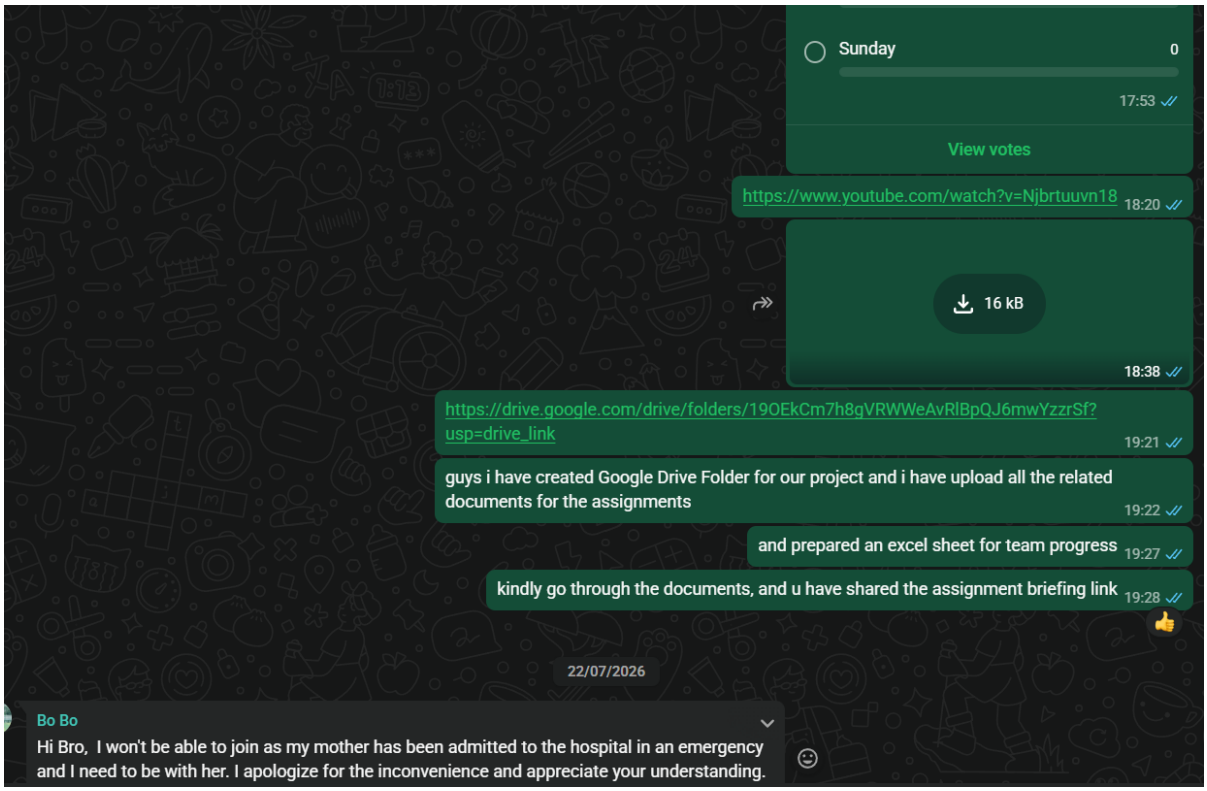
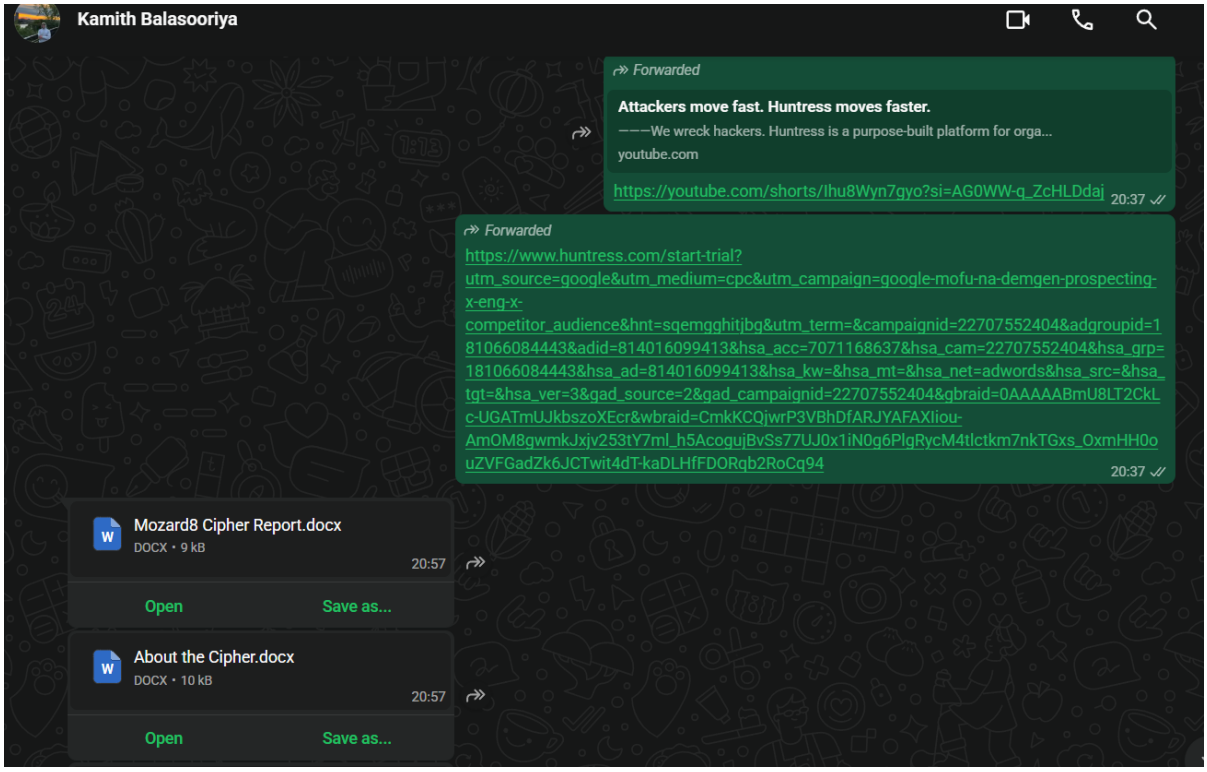
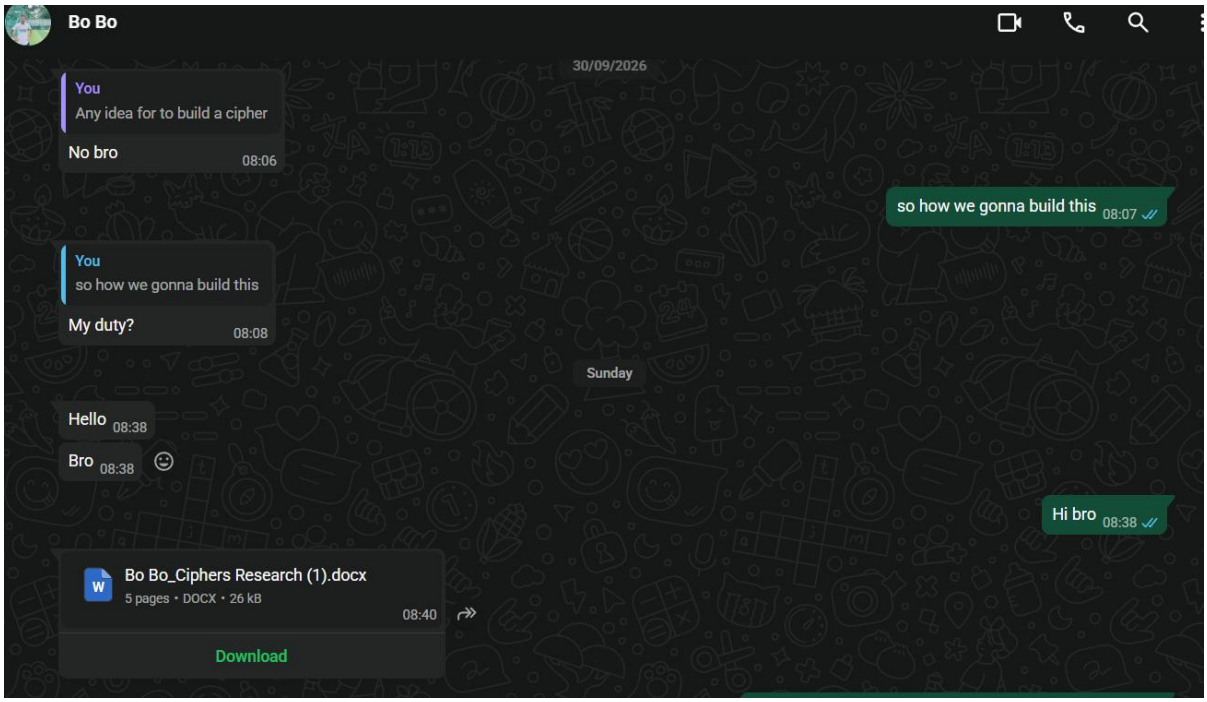


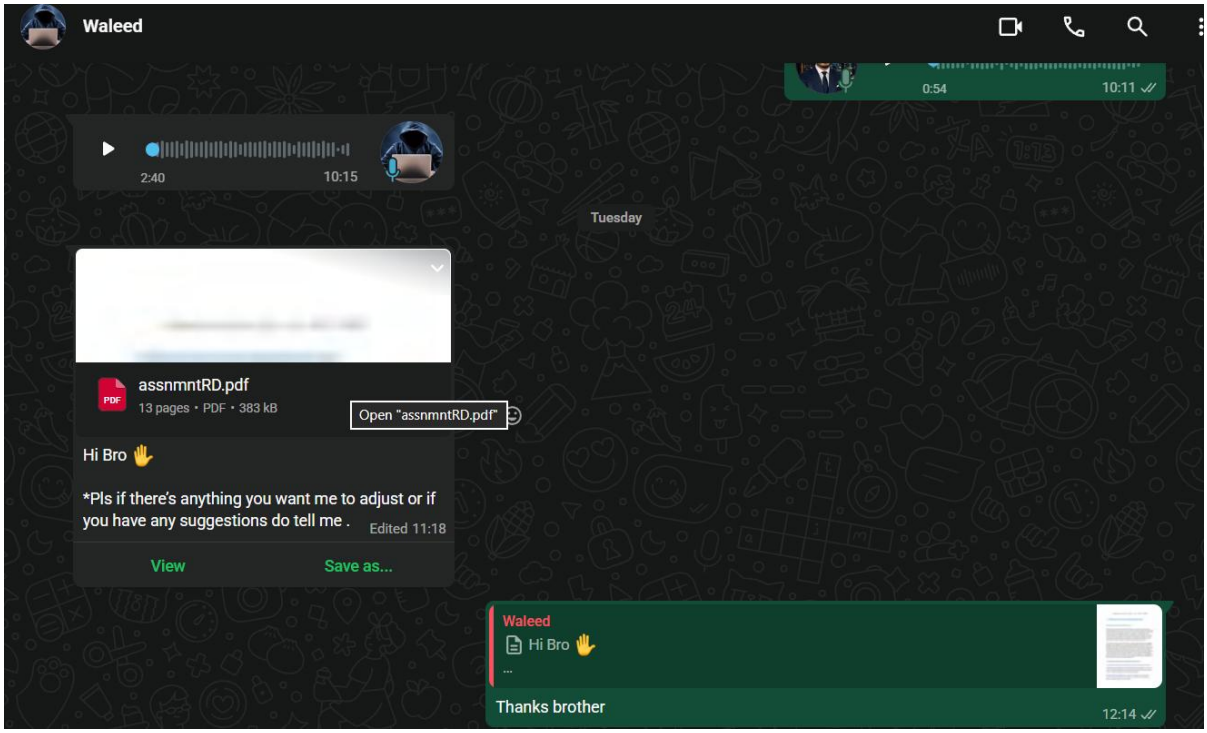
Figure 6 WhatsApp Group – Sc




Sunday 0
17:53
View votes
https://www.youtube.com/watch?v=Njbrtuuvn18 18:20
16 kB
18:38
https://drive.google.com/drive/folders/19OEkCm7h8gVRWWeAvRlBpQJ6mwYzzrSf?usp=drive_link 19:21
guys i have created Google Drive Folder for our project and i have upload all the related documents for the assignments 19:22
and prepared an excel sheet for team progress 19:27
kindly go through the documents, and u have shared the assignment briefing link 19:28
22/07/2026
Bo Bo
Hi Bro, I won't be able to join as my mother has been admitted to the hospital in an emergency and I need to be with her. I apologize for the inconvenience and appreciate your understanding.


Kamith Balasooriya
Forwarded
Attackers move fast. Huntress moves faster.
———We wreck hackers. Huntress is a purpose-built platform for orga...
youtube.com
https://youtube.com/shorts/Ihu8Wyn7gyo?si=AG0WW-q_ZcHLDdaj 20:37
Forwarded
https://www.huntress.com/start-trial?utm_source=google&utm_medium=cpc&utm_campaign=google-mofu-na-demgen-prospecting-x-eng-x-competitor_audience&hnt=sqemgghitjbg&utm_term=&campaignid=22707552404&adgroupid=181066084443&adid=814016099413&hsa_acc=7071168637&hsa_cam=22707552404&hsa_grp=181066084443&hsa_ad=814016099413&hsa_kw=&hsa_mt=&hsa_net=adwords&hsa_src=&hsa_tgt=&hsa_ver=3&gad_source=2&gad_campaignid=22707552404&gbraid=0AAAAABmU8LT2CkLc-UGATmUJkbszoXEcr&wbraid=CmkKCQjwrP3VBhDfARJYAFAXIiou-AmOM8qwmkJxjv253tY7ml_h5AcogujBvSs77UJ0x1iN0g6PlgRycM4tlctkm7nkTGxs_OxmHH0ouZVFGadZk6JCTwit4dT-kaDLHfFDORqb2RoCq94
20:37
Mozard8 Cipher Report.docx
DOCX • 9 kB
20:57
Open
Save as...
About the Cipher.docx
DOCX • 10 kB
20:57
Open
Save as...


Bo Bo
30/09/2026
You
Any idea for to build a cipher
No bro 08:06
so how we gonna build this 08:07
You
so how we gonna build this
My duty? 08:08
Sunday
Hello 08:38
Bro 08:38
Hi bro 08:38
Bo Bo_Ciphers Research (1).docx
5 pages • DOCX • 26 kB
08:40
Download


Waleed
0:54 10:11
2:40 10:15
Tuesday
assnmntRD.pdf
13 pages • PDF • 383 kB
Open "assnmntRD.pdf"
Hi Bro
*Pls if there's anything you want me to adjust or if you have any suggestions do tell me . Edited 11:18
View
Save as...
Waleed
Hi Bro
...
Thanks brother 12:14